To	Buzzebees Co., Ltd. — CRM PLUS project team
Cc	Easy POS — account manager (section 7 only)
From	Oshinei — project owner, CRM integration
Re	Your document "API CRM PLUS_full - 2026.xlsx" (10 sheets, specification versions 1–3, 2022–2025)

Proposal: integrate CRM PLUS through the Oshinei CRM API, with Buzzebees as the lead party and no change to the Easy POS application

Thank you for the CRM PLUS POS API specification. Your API requires the POS application to call Buzzebees at checkout, which would require development by Easy POS. We propose a model that needs no change to the POS application and no access by any third party to our POS database. Oshinei already reads completed sales from the Easy POS head-office database (HQ_Oshinei) into its own analytics application with a read-only account. We will add a CRM module to that application and publish an HTTPS API: Buzzebees reads bills from it, runs point earning, reversal and coupon tracking in CRM PLUS, and writes every result back to it. This letter sets out the model, the evidence from our sales data, the API specification, and what we ask of Buzzebees and Easy POS.

1. Summary

Model. Buzzebees is the lead party and builds a connector to the Oshinei CRM API. Every 5 minutes it reads bills that are new or changed, including voids. Members identify a bill by claiming the receipt in the LINE Official Account. Buzzebees credits points, reverses them when a bill is voided on any later day, marks coupons used when a cashier enters the code in Easy's existing "external coupon" function, and reports each of these results back to Oshinei through the API.

What changes. Oshinei builds and operates the API (section 4). Easy POS develops nothing and grants no database access; it only confirms how its coupon and receipt functions work (section 7).

Trade-off. Points arrive after the bill reaches the head office and the member claims it, not at the counter, and coupons are checked after use rather than before. Section 2.2 compares this with the earlier options.

Ref.	Request to Buzzebees	Priority
R1	Confirm that Buzzebees will build and operate the connector to the Oshinei CRM API and credit points from it	Blocking
R2	Provide a receipt-claim flow in the LINE Official Account, with the anti-abuse checks in section 6	Blocking
R3	Reverse points automatically when a claimed bill is voided, on any later day	Blocking
R4	Issue unique coupon codes usable as Easy external coupons, and mark them used from the API	Blocking
R5	Let members exchange points for coupons in LINE (no point burning at the counter)	Before pilot
R6	Write every claim, credit, reversal and coupon result back to the API within 5 minutes	Before pilot
R7	Send a daily summary through the API and resolve reconciliation differences	Before pilot
R8	Sign a data processing agreement and store only the data listed in section 9	Before pilot
R9	Optional: credit points automatically when the cashier records the member's phone on the bill	Optional
R10	Revoke the tokens contained in the specification file and issue Oshinei credentials separately	Before pilot

We request a written reply to R1–R10 within 10 business days (proposed), followed by a joint meeting.

Contents

1 Summary 1

2 Proposed integration model 4

3 Evidence from our sales database 6

4 Oshinei CRM API specification 9

5 Process flows 15

6 Requests to Buzzebees (R1–R10) 17

7 Requests to Easy POS (E1–E4) 19

8 Items Oshinei will confirm (D1–D6) 20

9 Security and data protection 21

10 Pilot, acceptance tests and next steps 22

A Appendix A — Use of the CRM PLUS API in this model 24

B Appendix B — Data sources and method 25

2. Proposed integration model

Sales flow from branch POS terminals to the head-office database as they do today. Oshinei's application reads them with its existing read-only account and publishes them through the CRM API. Buzzebees reads from the API, runs all loyalty logic on its own platform, and writes the results back. No party other than Oshinei connects to the POS database.

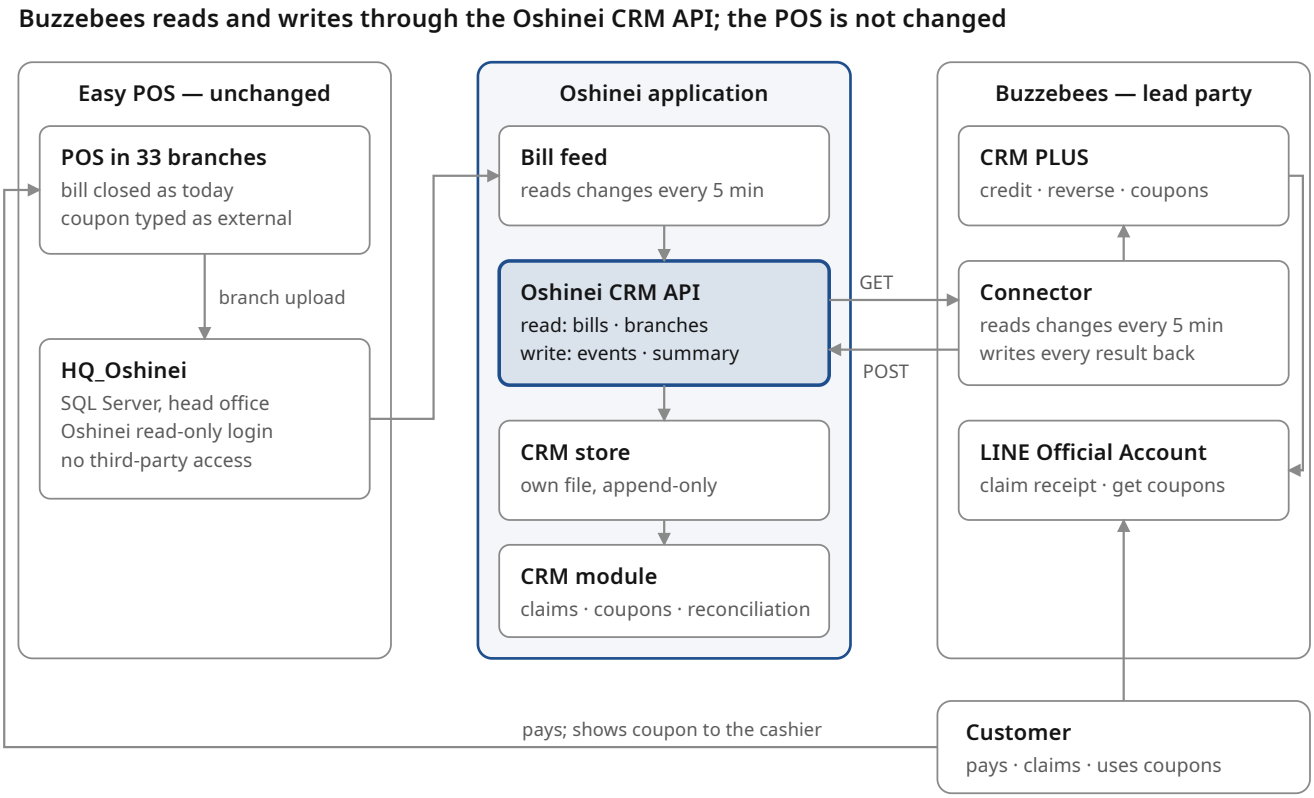


Figure 1. Proposed model. Only Oshinei reads the POS database. Buzzebees uses the HTTPS API in both directions.

2.1 Responsibilities

Party	Responsibilities
Buzzebees	Build and operate the connector to the Oshinei CRM API; receipt claim, crediting, reversal and coupon tracking in CRM PLUS; LINE Official Account; write every result back (R6) and send a daily summary (R7); data protection as a processor
Oshinei	Build and operate the bill feed and the CRM API to the service levels in section 4.9; keep the results in a separate store; CRM module for head office and branches; business rules (section 8); cashier instructions; reconciliation review; acceptance
Easy POS	No development and no database access for third parties. Confirm the coupon, receipt and upload items in section 7

2.2 Comparison with the earlier options

Topic	Oshinei CRM API (this revision)	Direct database read (Rev. 1)	Real-time POS (your specification)
Development	Buzzebees connector; Oshinei API	Buzzebees connector	Easy develops the POS side through HO_CRM
Easy POS work	Confirmations only	Views, login, network access	Development and roll-out to all branches
Third-party access to the POS database	None	Read-only login to the HQ server	None
Interface	HTTPS and JSON, change cursor	SQL views, time watermark	CRM PLUS API at checkout
Results to Oshinei	Every event, within 5 minutes	Daily report	Through the POS
Points available	After upload to HQ and claim (minutes to hours)	Same	Immediately; printed on the receipt
Void on a later day	Always reversed	Always reversed	`/earn/cancel` : same day only
Coupon check	After use; second use reported	Same	Before use at the counter
Counter workload	Type a coupon code when used	Same	Scan member QR or enter phone on every bill

The real-time model remains possible later. The API would still be needed for reconciliation, so this work is not wasted if we move to real time.

3. Evidence from our sales database

Figures from HQ_Oshinei, January–September 2026 unless stated. They size the data the API will carry and justify the rules in sections 4–6.

3.1 Volume

500,905
finalised bills, Jan–Sep 2026

≈1,650
bills on an average day (Sep 2026)

452
bills in the busiest hour (12 Aug 2026, 13:00)

33
trading branches (Sep 2026), one POS terminal each in most

Month 2026	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep
Finalised bills	69,475	63,003	50,381	54,507	61,337	47,754	50,806	54,314	49,328
Trading branches	27	27	27	29	30	31	32	33	33
Voided bills	295	233	263	254	258	275	251	317	337

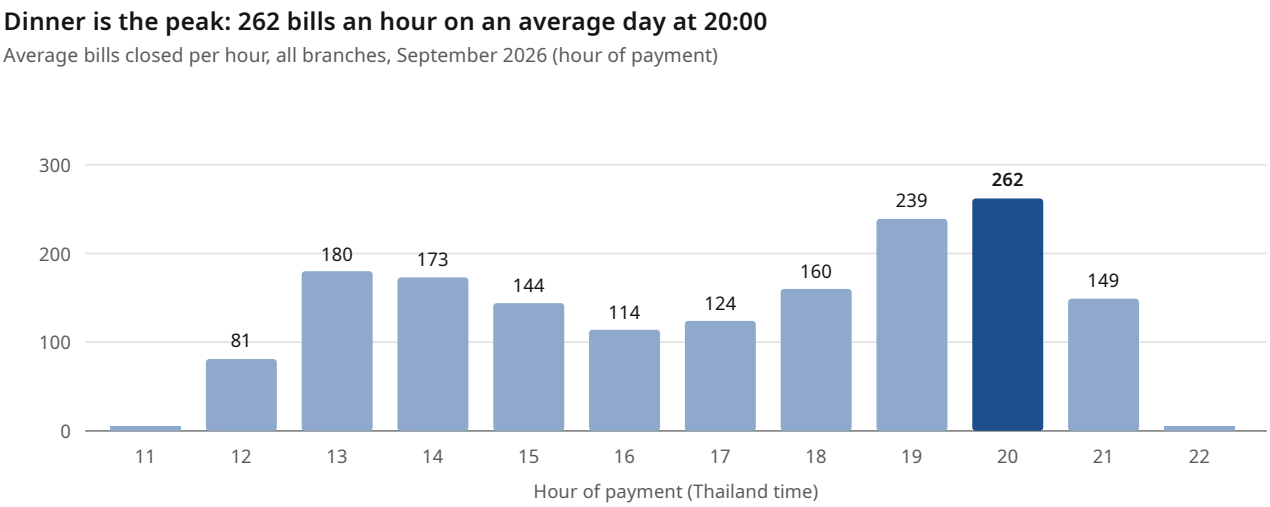


Figure 2. Average hourly bill volume, September 2026.

Implication for polling. At the busiest hour, a 5-minute poll of `/v1/bills/changes` returns about 40 new or changed bills. Oshinei's feed reads only changed rows from HQ_Oshinei, so the extra load on the database is small.

3.2 No loyalty data exists today

No bill in 2026 carries points (`Sale.EarnPoint` and `Sale.RedeemedPoint` are zero on all 500,905 bills and on all bills of 1–26 October). The Easy membership tables (`MemberCard`, `CustPoint`, `CustomerLoyalty`, `LoyaltyRank` and others) are empty. The `Customer` table (7,128 rows) holds tax-invoice records, not members. The member base therefore starts in CRM PLUS with no migration.

3.3 How a bill is identified

In September 2026 the 49,665 finalised bills had 49,665 unique branch + `SaleNum` pairs but only 33,542 distinct `SaleId` values. The bill key in this proposal is therefore **branch code + SaleNum**. `SaleNum` is printed on the receipt, contains the business date and a daily sequence, and is predictable; section 6 explains why a claim must also match the amount.

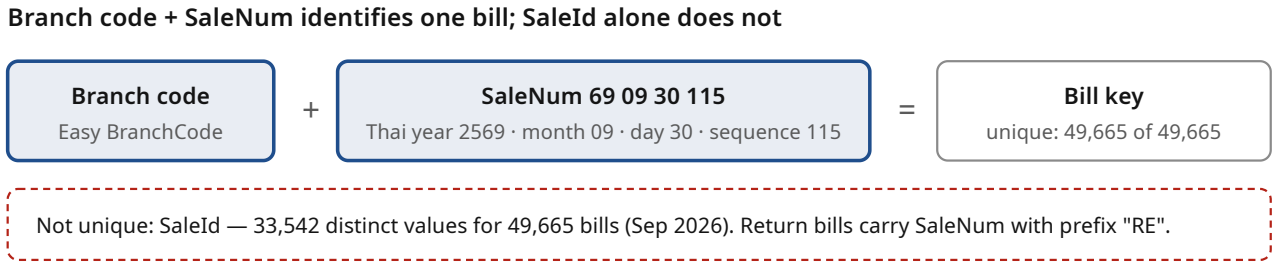


Figure 3. Bill identifier in Easy POS, used as the claim and credit key in this proposal.

3.4 Voids happen up to 19 days after the sale

Of 2,482 voided bills, 2,446 (98.5%) were voided on the day of sale and **36 on a later day, up to 19 days after**. Every voided bill updates `Sale.UpdateDate` to the void time or later (2,481 of 2,482), so Oshinei's feed finds voids by reading changed rows and publishes the bill again with status `voided`. A credit given on day 0 can therefore always be reversed, unlike the same-day limit of `/earn/cancel`.

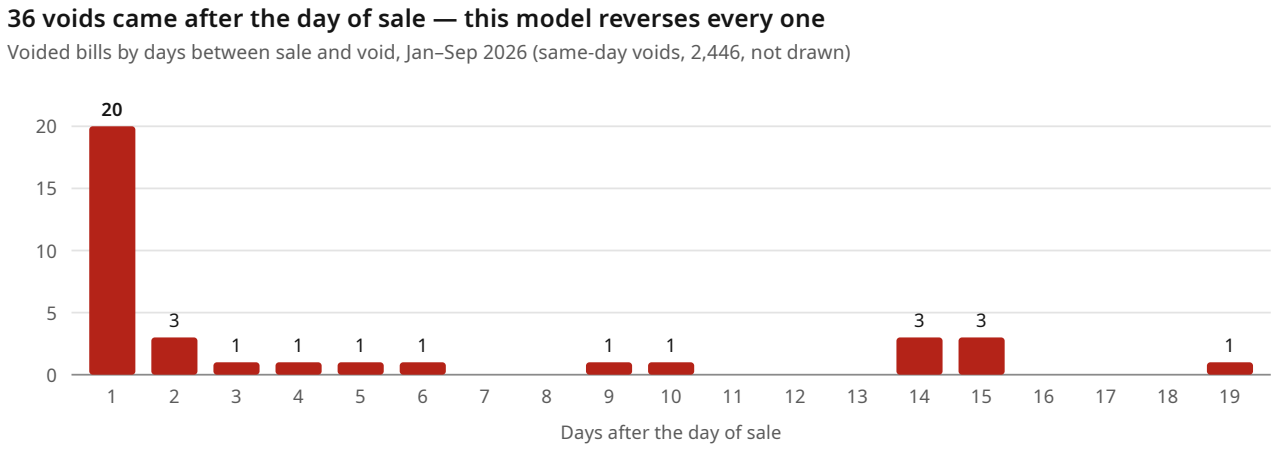


Figure 4. Voids made after the day of sale, by delay in days. Total 36; the longest delay was 19 days.

3.5 Data reaches the head office during the day

Easy records each branch's last upload in `Branch.LastSync`. On 27 October 2026 at 11:17, 16 of 31 active branches had uploaded within the previous 30 minutes, 14 within the previous 24 hours, and 1 had not uploaded for more than a day. A bill closed at 10:38 was already readable at 11:13. Upload timing differs by branch, so we will measure it for one week during the pilot and agree a target with Easy (E3). `/v1/branches` gives each branch's last upload time, and both parties alert when a branch has not uploaded for 3 hours during trading (R7).

16 / 31	**14 / 31**	**1 / 31**	**35 min**
branches uploaded in the last 30 minutes (27 Oct, 11:17)	uploaded within the last 24 hours	no upload for more than 24 hours	bill 10:38 readable at HQ by 11:13

3.6 External coupons are already used at the counter

Cashiers already enter external coupon numbers into Easy. They are stored in `CouponCardOutsideSystemUsed` with the coupon series, coupon number, bill, branch, discount and time. From January to September 2026 there were **38,156 uses of 20,459 distinct coupon numbers**. **280 numbers were used on more than one bill** (4,506 uses in total), because only 3 of Easy's 136 coupon series enforce unique numbers (`IsUniqueCouponNumber`). On 26 October 2026, coupon number 2499 of one series was used on two bills in the same branch 77 minutes apart. Buzzebees codes must therefore be unique and checked after use (R4).

4. Oshinei CRM API specification

Version 1, proposed for the pilot. Field names and paths are final unless Buzzebees asks for a change in its reply. Oshinei builds the source of every field from HQ_Oshinei with its existing read-only account.

4.1 Conventions

Item	Rule
Host	A dedicated HTTPS host, separate from the Oshinei analytics application. The address is sent with the credentials. Paths start with `/v1`
Transport	TLS 1.2 or later. Requests are accepted only from the Buzzebees IP addresses on the allow-list
Authentication	Header `Authorization: Bearer <token>`. Two tokens: one with read scope and one with write scope. Tokens never appear in URLs or logs and are rotated every 90 days
Format	JSON in UTF-8. Times in ISO 8601 with the Thailand offset (`2026-09-30T19:05:12+07:00`). Dates as `YYYY-MM-DD`. Money as strings with two decimals (`"1999.00"`) in Thai baht
Paging	Cursor-based. `limit` up to 500. The response gives `next_cursor` and `has_more`
Idempotency	Every write carries a unique `event_id`. Sending the same `event_id` again stores nothing new and returns it as a duplicate
Rate limit	120 requests per minute per token. Above that the API returns `429` with `Retry-After`
Changes	New fields may be added at any time; clients ignore fields they do not know. Changes that break clients come as `/v2` with 60 days' notice
Environments	A sandbox with test data from the start of the build; production from the pilot

4.2 Endpoints

Method	Path	Scope	Purpose
GET	`/v1/bills/changes`	read	Bills that are new or changed since a cursor: paid, voided and returned
GET	`/v1/bills/{bill_key}`	read	One bill, to check a claim at once
GET	`/v1/branches`	read	Branch list with the last upload time of each branch
GET	`/v1/status`	read	Freshness of the bill feed and state of the service
POST	`/v1/events`	write	Results: claims, credits, reversals and coupons, up to 200 per request
PUT	`/v1/daily-summary/{date}`	write	Totals for one business date
GET	`/v1/reconciliation/{date}`	read	Oshinei's comparison of the events and summary with sales for that date

4.3 The bill object

Field	Type	Meaning
bill_key	string	Branch code + "-" + `SaleNum`. Unique bill identifier; the key for claims, credits and coupons (section 3.3)
branch_code	string	Easy branch code, as in `/v1/branches`
sale_num	string	Number printed on the receipt
business_date	date	Date of the sale
closed_at	datetime	Time the bill was paid
guests	integer	Number of guests
sales_channel	string	`dine_in`, `takeaway`, `grab`, `lineman` or `other`. Needed for decision D2
subtotal	money	Amount before discount and VAT
discount	money	Bill discount
service_charge	money	Service charge
vat	money	VAT
total	money	Amount paid, including VAT. The amount a member enters when claiming
status	string	`paid`, `voided` or `returned`
voided_at	datetime or null	Time of the void, if any
payments	list	`method` (`cash`, `card`, `qr`, `transfer`, `platform`, `other`) and `amount`. No card number or holder name
coupons	list	External coupons of the Buzzebees series entered on the bill: `series_code`, `number`, `discount`
member_phone	string or null	Phone typed by the cashier. Present only if R9 is adopted
changed_at	datetime	When Oshinei last saw a change to the bill

Not in the API: the `Customer` table (tax-invoice records), cashier names, card numbers, holder names and bill notes.

4.4 GET /v1/bills/changes

Returns bills in the order Oshinei saw them change. A bill appears again each time it changes, for example when it is voided days later. The connector always applies the latest state of each `bill_key`.

Parameter	Meaning
cursor	The `next_cursor` from the previous response. Omit it on the first call
from_date	First call only, or to replay after an incident: start from this business date
limit	1–500, default 500

```
GET /v1/bills/changes?cursor=c_000184120&limit=500

200 OK
{
  "items": [
    {
      "bill_key": "0001-690930115",
      "branch_code": "0001",
      "sale_num": "690930115",
      "business_date": "2026-09-30",
      "closed_at": "2026-09-30T19:05:12+07:00",
      "guests": 4,
      "sales_channel": "dine_in",
      "subtotal": "1868.22", "discount": "0.00", "service_charge": "0.00",
      "vat": "130.78", "total": "1999.00",
      "status": "paid", "voided_at": null,
      "payments": [ { "method": "qr", "amount": "1999.00" } ],
      "coupons": [],
      "member_phone": null,
      "changed_at": "2026-09-30T19:21:40+07:00"
    }
  ],
  "next_cursor": "c_000184233",
  "has_more": false
}
```

The cursor never skips a change, so the connector needs no time overlap and no nightly re-scan. It stores `next_cursor` only after it has processed the page, and calls again at once while `has_more` is true.

4.5 GET /v1/bills/{bill_key}, /v1/branches and /v1/status

Endpoint	Response
`/v1/bills/{bill_key}`	The bill object, or `404 bill_not_found` if the bill has not reached the head office yet or does not exist. For checking a claim at once; if not found, the claim is held (R2). A token that receives more than 300 not-found responses in an hour is suspended, to stop receipt numbers being guessed
`/v1/branches`	`branch_code`, `name`, `active`, `last_upload_at` (when the branch last sent data to the head office), `last_bill_at`
`/v1/status`	`state` (`ok`, `degraded`, `maintenance`), `feed_read_at` (last successful read of HQ_Oshinei), `feed_lag_seconds`

4.6 POST /v1/events

Buzzebees reports every result through this endpoint (R6). Oshinei keeps events as they are received and never changes them; a correction is sent as a new event.

Field	Type	Meaning
event_id	string, ≤ 64	Unique for ever. The idempotency key
type	string	One of the types in the next table
occurred_at	datetime	When it happened in CRM PLUS
bill_key	string	The bill concerned, where there is one
member_id	string	CRM PLUS internal member identifier. No phone, name or LINE identifier
points	integer	Positive for a credit, negative for a reversal
coupon_code, coupon_value	string, money	For coupon events
reason	string	For rejections and reversals (codes below)

Type	Required fields	When
`claim.accepted`	bill_key, member_id	A claim passed all checks
`claim.held`	bill_key, member_id	The bill is not in the API yet; re-checked for up to 48 hours
`claim.rejected`	bill_key, member_id, reason	`not_found`, `amount_mismatch`, `already_claimed`, `too_old`, `voided`, `daily_limit`
`points.credited`	bill_key, member_id, points	Points given for a bill
`points.reversed`	bill_key, member_id, points, reason	Bill `voided` or `returned`
`coupon.issued`	member_id, coupon_code, coupon_value	From a campaign or in exchange for points
`coupon.used`	coupon_code, bill_key	First use seen on a bill
`coupon.restored`	coupon_code, bill_key	The bill was voided; the coupon is unused again
`coupon.duplicate_use`	coupon_code, bill_key	The same code seen on a second bill

```
POST /v1/events
{
  "events": [
    { "event_id": "bzb-7f3a91c2", "type": "claim.accepted",
      "occurred_at": "2026-09-30T20:30:02+07:00",
      "bill_key": "0001-690930115", "member_id": "M0004821" },
    { "event_id": "bzb-7f3a91c3", "type": "points.credited",
      "occurred_at": "2026-09-30T20:30:02+07:00",
      "bill_key": "0001-690930115", "member_id": "M0004821", "points": 199 }
  ]
}

200 OK
{ "accepted": 2, "duplicates": 0, "rejected": [] }
```

Items in `rejected` carry the `event_id` and an error code, for example `unknown_bill_key`. The rest of the batch is stored.

4.7 PUT /v1/daily-summary/{date} and GET /v1/reconciliation/{date}

By 06:00 each day Buzzebees sends the totals for the previous business date: claims accepted, held and rejected; points credited and reversed; coupons issued, used, restored and used twice; new members and total members. Sending the summary again replaces it, and Oshinei keeps every version.

Oshinei then compares the summary with the events received and with sales in the bill feed. `/v1/reconciliation/{date}` returns `matched` or `mismatch` with a list of differences (item, Buzzebees value, Oshinei value). Differences are resolved within 3 business days (R7).

4.8 Errors

HTTP	Code	Meaning and action
400	`invalid_request`	Malformed request. Fix it; do not retry as it is
401	`unauthorized`	Missing, expired or revoked token
403	`forbidden`	Wrong scope, or an IP address not on the allow-list
404	`bill_not_found`	Bill not in the API yet. Hold the claim
409	`event_conflict`	An `event_id` already stored with different content. Investigate; never reuse identifiers
429	`rate_limited`	Wait for `Retry-After`
503	`maintenance`	Retry with back-off. Events are kept by Buzzebees and resent

Error body: `{ "error": "bill_not_found", "message": "..." }`.

4.9 Service levels from Oshinei (proposed)

Item	Commitment
Freshness	A bill appears in the API within 10 minutes of reaching HQ_Oshinei. How fast it reaches HQ depends on the branch upload (section 3.5, E3)
Availability	99.5% a month, measured 09:00–24:00. Planned maintenance 02:00–05:00 with 2 days' notice
Outage	Nothing is lost: the feed resumes from the cursor, and Buzzebees resends unsent events with the same `event_id`
Events kept	Stored in a database separate from the sales data, backed up daily to separate storage, kept 5 years
Support	A named technical contact; incidents acknowledged within 1 hour between 09:00 and 22:00

5. Process flows

5.1 Earning by receipt claim

A claim earns points only when it matches a paid bill that has not been claimed before

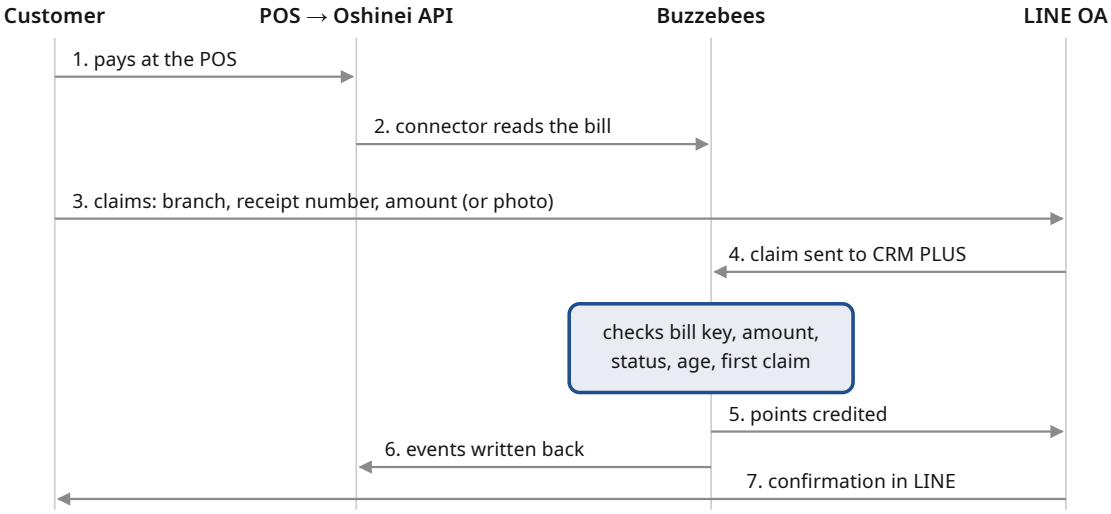


Figure 5. Receipt claim. If the bill has not reached the API yet, the claim is held and re-checked automatically for up to 48 hours.

1. The customer pays as today. The receipt shows the branch and receipt number (`SaleNum`).
2. The bill reaches HQ_Oshinei through Easy's existing branch upload, Oshinei's feed reads it, and it appears in `/v1/bills/changes`.
3. In the LINE Official Account the member enters the branch, receipt number and amount paid, or uploads a photo of the receipt.
4. Buzzebees checks that the bill exists, is `paid`, matches the amount, is within the claim window and has not been claimed. It may call `/v1/bills/{bill_key}` to check at once. If the bill is not there yet, the claim is held and re-checked for up to 48 hours.
5. CRM PLUS credits the points according to the earn rules (D2).
6. The connector writes `claim.accepted` and `points.credited` to `/v1/events`.
7. The member receives a confirmation in LINE.

5.2 Reversal when a bill is voided

A void on any later day removes the points credited for that bill

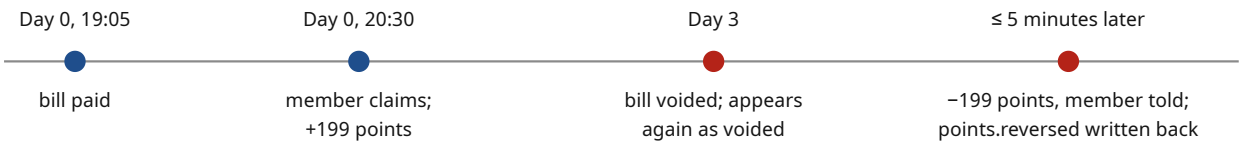
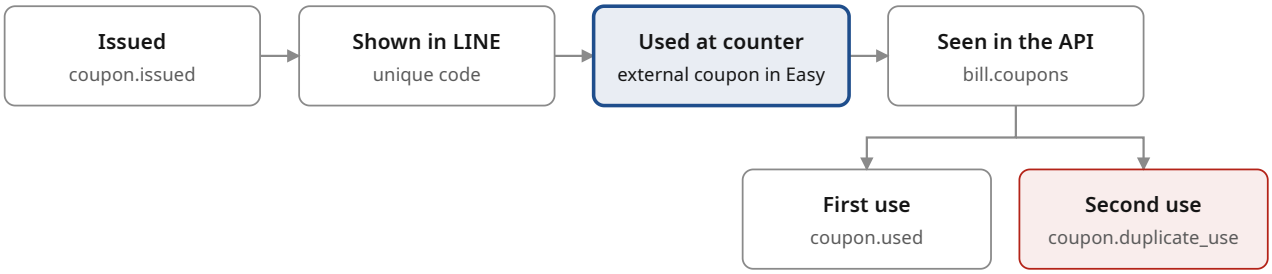


Figure 6. Reversal timeline. In 2026 the latest void was 19 days after the sale; the change feed delivers it whenever it happens.

If the member has already spent the points, CRM PLUS applies the rule chosen by Oshinei (D4): a negative balance, or deduction from the next credit. A bill that becomes `returned` is treated the same way.

5.3 Coupon lifecycle

A coupon is issued in LINE, used at the counter, and marked used from the API



If the bill is voided, the coupon is restored to unused and coupon.restored is written back.

Figure 7. Coupon lifecycle. Each step is reported to Oshinei as an event.

1. CRM PLUS issues a coupon with a unique number, from a campaign or in exchange for points (R5).
2. The member shows the coupon in LINE. The cashier applies it in Easy as an external coupon of the Buzzebees series and types the number.
3. The bill reaches the API with the coupon in `coupons`. The connector marks the coupon used in CRM PLUS and writes `coupon.used`.
4. A second use of the same number is not blocked at the counter. The connector writes `coupon.duplicate_use`; the Oshinei CRM module shows it to head office the same day with the branch and bill.
5. If the bill is voided, the connector restores the coupon and writes `coupon.restored`.

Because use is checked after the event, we will limit Buzzebees coupons to modest values during the pilot (D5) and ask Easy whether the coupon series can block reuse at the counter (E1).

6. Requests to Buzzebees (R1–R10)

R1 — Build and operate the connector to the Oshinei CRM API Blocking

Request	Confirm that Buzzebees will build, host and operate the connector that reads `/v1/bills/changes` every 5 minutes and credits, reverses and marks coupons in CRM PLUS, following section 4 (rate limit, fixed IP addresses, no guessing of bill keys). Please state the CRM PLUS functions it will use (Appendix A), the effort, whether it is within the standard fee or charged separately, and the delivery date.
Why	This makes Buzzebees the single lead party, removes POS development by Easy, and keeps our POS database closed to third parties.

R2 — Receipt claim in the LINE Official Account Blocking

Request	A claim screen that accepts branch, receipt number and amount paid, or a receipt photo. A claim succeeds only if all of the following hold: • the bill key exists in the API with status `paid`; • the amount matches `total` within 1 THB; • the business date is within the claim window (7 days proposed, D3); • the bill key has not been claimed before; • the member has made no more than 5 claims that day (proposed). Claims for bills not yet in the API are held and re-checked for up to 48 hours.
Why	`SaleNum` is sequential within a day (section 3.3), so a receipt number alone could be guessed. The amount match and daily limit prevent claiming other customers' bills.

R3 — Automatic reversal on void Blocking

Request	When a claimed bill appears again with status `voided` or `returned`, reverse the credit within one polling cycle, on any day, notify the member and write `points.reversed`. Receiving the same change twice must not reverse twice.
Why	36 bills in 2026 were voided after the day of sale, up to 19 days later (section 3.4). `/earn/cancel` in your specification allows same-day reversal only.

R4 — Coupon codes for Easy external coupons Blocking

Request	Issue coupons with unique numeric codes of a fixed length that a cashier can type into Easy (length to be confirmed by Easy, E1). Mark a coupon used when it appears in a bill's `coupons`; restore it when the bill is voided; report every second use of a code. If Easy requires codes to be loaded in advance, provide them as a file in the format Easy specifies.
Why	280 external coupon numbers were used on more than one bill in 2026, because Easy enforces unique numbers in only 3 of 136 series (section 3.6).

R5 — Exchanging points for coupons in LINE Before pilot

Request	Members burn points only in LINE, by exchanging them for a coupon (R4). No point burning at the counter. Please confirm the exchange catalogue is configurable by Oshinei.

R6 — Write every result back Before pilot

Request	Send each claim, credit, reversal and coupon result to `/v1/events` within 5 minutes (section 4.6), with a unique `event_id` and the CRM PLUS member identifier only. On any failure, resend with the same `event_id` until it is accepted.
Why	CRM PLUS keeps the member balances. Oshinei needs each result next to its sales to see the cost of the programme by branch, follow up duplicate coupons the same day, and reconcile.

R7 — Daily summary and reconciliation Before pilot

Request	Send `/v1/daily-summary/{date}` by 06:00 for the previous business date (section 4.7). Read `/v1/reconciliation/{date}` and resolve differences within 3 business days. Alert Oshinei when `/v1/status` shows the feed more than 30 minutes behind during trading, or a branch with no upload for 3 hours.

R8 — Data processing agreement Before pilot

Request	Sign a data processing agreement with Oshinei as controller. Store from the API only bill keys, amounts, dates, branch codes, sales channel, payment methods, coupon uses and, if R9 is adopted, the member phone. Delete or anonymise sales data older than 24 months (proposed). State where data is hosted.

R9 — Optional: earning from a phone recorded on the bill Optional

Request	If Easy confirms the cashier can type the member's phone into an existing field (`CustRefNum` , empty on all 2026 bills; E4), credit points automatically for a registered member whose phone matches `member_phone` , without a claim.
Note	This adds counter work and needs the member's consent in LINE. Oshinei will decide after the pilot (D1).

R10 — Credentials in the specification file Before pilot

Request	The specification file contains merchant tokens in clear text, including tokens labelled "TMN only" and "All Except TMN". Please confirm whether they are valid in any environment, revoke them if so, and send any credentials Oshinei needs through a separate secure channel.

7. Requests to Easy POS (E1–E4)

Confirmations only. Easy POS is not asked to develop anything, create views or logins, or open network access. Oshinei sends these questions to Easy separately.

Ref.	Request	Why
E1	Set up a coupon series for Buzzebees codes as external coupons. Confirm the code length accepted, whether codes must be loaded in advance and, if so, whether they can be imported in bulk, and whether `IsUniqueCouponNumber` blocks reuse at the counter	R4; duplicate use today (section 3.6)
E2	Confirm that the receipt prints the branch name and `SaleNum` legibly; optionally enable the receipt QR feature already assigned to 35 branches	Members claim from the receipt
E3	Confirm how often each branch uploads to the head office and whether the interval can be shortened to 15 minutes	Points arrive only after upload (section 3.5)
E4	Confirm whether the cashier screen allows entering a value into `CustRefNum` without development	Input to R9

8. Items Oshinei will confirm (D1–D6)

Ref.	Decision	Current preference
D1	Member identification	Receipt claim first; phone on bill (R9) after the pilot
D2	Earn base and rate: `total` including VAT and service charge, or amount before VAT; whether Grab and LINE MAN bills earn	To be confirmed
D3	Claim window	7 days from the business date
D4	Reversal after points are spent: negative balance or deduction from the next credit	To be confirmed
D5	Maximum coupon value during the pilot	To be confirmed
D6	Whether item-level campaigns are needed (adds bill items to the API)	Not in the pilot

9. Security and data protection

Area	Rule
POS database	Closed to third parties. Only Oshinei's existing read-only account reads HQ_Oshinei; the API never writes to it
Least privilege	Separate read and write tokens; the API exposes only the fields in section 4.3
Data excluded	`Customer` table, cashier names, card number, holder name, CVV and expiry, bill notes
Network	HTTPS with TLS 1.2 or later; Buzzebees IP addresses on an allow-list; Oshinei can suspend access at any time
Credentials	Exchanged through a separate secure channel; rotated every 90 days; never in documents, URLs or logs
Guessing receipts	Not-found lookups limited per token (section 4.5); claims need the amount as well as the receipt number (R2)
Personal data	Member identity stays in CRM PLUS. Events carry only the CRM PLUS member identifier. The phone is in the API only if R9 is adopted with member consent. Buzzebees acts as processor under the Personal Data Protection Act B.E. 2562 (R8)
Audit	Oshinei logs every request (time, token, path, result) for 12 months; Buzzebees logs its calls and provides the log on request. Events are kept unchanged

10. Pilot, acceptance tests and next steps

10.1 Acceptance tests (pilot branch)

- [ ] A paid bill claimed with the correct amount earns points once; a second claim of the same bill is rejected.
- [ ] A claim with a wrong amount, an expired date or an unknown receipt is rejected with a clear reason.
- [ ] A claim made before the bill reaches the API is held and credited automatically once the bill arrives.
- [ ] A claimed bill voided on day 0 and another voided on day 3 are both reversed within one polling cycle.
- [ ] A coupon used once is marked used; a second use of the same code is reported the same day; a coupon on a voided bill is restored.
- [ ] Reading the change feed from a saved cursor twice causes no duplicate credit or reversal.
- [ ] Sending the same events twice stores them once.
- [ ] After a planned API outage of one hour, the connector catches up from its cursor and resends its events with nothing lost.
- [ ] The reconciliation for each day of the test week shows `matched`.

10.2 Next steps

No.	Step	Owner	Timing (proposed)
1	Written reply to R1–R10 with effort, cost, date and any change to the API	Buzzebees	Within 10 business days
2	Written reply to E1–E4	Easy POS	Within 10 business days
3	Joint meeting; approve the API and business rules	Oshinei (convenor)	After steps 1–2
4	Sandbox API with test data and credentials issued	Oshinei	3 weeks after the meeting
5	Connector build and tests against the sandbox	Buzzebees	To be agreed
6	Pilot in one branch for 4 weeks, upload timing measured in week 1, go/no-go	Oshinei	To be agreed

Yours faithfully,

For Oshinei	Acknowledged by Buzzebees
Name:	Name:
Position: Project owner, CRM integration	Position:
Date:	Date:

Appendix A — Use of the CRM PLUS API in this model

The endpoints in your specification are not called by the POS in this model. Buzzebees may use them internally from the connector; that choice is Buzzebees'.

Endpoint (your sheet)	Use in this model	Note
`GET /pos/profile` (Profile)	Not used	Members are identified in LINE, not at the counter
`POST /pos/earn_direct` (Earn via user id)	Possible, by connector	Credit a known member after a successful claim; needs a reference to prevent double credit
`POST /pos/earn` (Earn)	Possible, by connector	For R9 only (credit by phone)
`POST /earn/cancel` (Refund point)	Not sufficient	Same-day limit; R3 needs reversal on any day
`POST /wallet/pay` (Redeem, Check code)	Possible, by connector	Mark a coupon used after it appears in a bill's `coupons`
Error Code, url	For Buzzebees	Internal to the connector

Observations on the specification file (for example the `points` field missing from the Earn-via-user-id table but present in its sample, and the Earn-direct-point sample calling `/earn_direct`) are not repeated here because the POS no longer calls these endpoints.

Appendix B — Data sources and method

Item	Detail
Source	Easy POS head-office database HQ_Oshinei (Microsoft SQL Server), Oshinei read-only account — the same account the bill feed will use — queried 14 and 27 October 2026
Period	1 January – 30 September 2026 unless stated; points checked to 26 October 2026; upload status at 11:17 on 27 October 2026
Finalised bill	`IsFinalize = 1 AND Active = 1 AND ISNULL(IsReturn,0) = 0`
Voided bill	`IsFinalize = 1 AND Active = 0`; delay = calendar days from `SaleDate` to `DeleteDate`; `UpdateDate ≥ DeleteDate` on 2,481 of 2,482
Hourly load	Hour of `SaleDate` (payment time), September 2026 average; busiest hour over July–September
Bill identifier	Distinct (BranchId, SaleNum) and distinct SaleId over finalised bills, September 2026
Upload status	`Branch.LastSync` for active branches with sales since 20 October 2026
External coupons	`CouponCardOutsideSystemUsed`, `UsedDate` in January–September 2026; reuse = same series and number on more than one bill; `CouponSeries.IsUniqueCouponNumber`
Bill example	The bill in section 4.4 is illustrative; its amounts are not a real bill

Prepared by Oshinei. This revision (Rev. 2) replaces Rev. 1 dated 27 October 2026, which proposed a direct read of the database, and the original letter, which proposed the real-time model.